The Thiele Machine

A Mathematical Reference

Devon Thiele

Before the specification. I don't do math by hand. I think in shapes, directions, and connections, and my part of this work was to see where things had to fit, to direct, and to refuse what didn't fit. This document is the translation. Language models wrote the statements, and I read every one to make sure it says what the shape says. Every result here is argued on the page: in the monograph, which each result names ("Monograph, Theorem 2.3" means that theorem of *The Thiele Machine: No Free Insight*), or in a short proof printed here. Where a result is someone else's, the source is named instead. Nothing here asks you to take a result on anyone's word, a program's included. So nothing below is in my voice, on purpose. It's the reference layer: exact statements, exact premises, and where each one lives. If you want the pictures, the monograph starts with them. If you want to know exactly what is claimed, it's here.

The Thiele Machine is an abstract model of computation, and an argument made with it about what an account of computation should preserve.

It is a machine in the sense a Turing machine is one: a mathematical object. No device is meant. The model starts from a set of states, moves, a step, a price on each move, a record that can be compared, and one rule, the toll: a move that takes the record out of the down-set of where it stood pays at least one. Any system with those parts is a priced record system; a Thiele machine is one whose record some run raises, from a designated start, and no move lowers. A yes-or-no reading is the smallest record, and most of what follows is stated on it. A reading no move carries out of its down-set is flat, and the toll asks nothing of it: a clock, a reversible process and every ordinary deterministic machine, read through a reading their moves never raise, are flat; each, once its states can be tested for equality and it has a run that leaves its start, is the shadow of a member. Forget the prices and the record of any Thiele machine and what is left is one of them. The same machine read differently need not be flat: a computation that halts, read through whether it has halted with the halted state stepping to itself, has a reading that climbs and never falls and a merge at the halt, so with its step priced it is a Thiele machine on its own states. What the usual formalism leaves out is the price, not the record. The results below are about what is kept and what keeping it costs. The latch, two states and one priced move, shows the definition can be met. It is not the model.

The structural question is what a total deterministic process needs to raise a reading and keep it: a merge along the run or endlessly fresh states. A physical price needs separate hypotheses. Landauer bounds depend on the physical model and input distribution; finite-time entropy-production bounds here concern specified thermal stochastic dynamics, not every physical state change. Certification, a record that a claim was checked and closed, is the sharpest case. It is one example of a wider question about structural state, observation and cost. A window, a function from states to what can be seen of them (Chapter 5), can lose distinctions that the full state keeps, such as the record and the ledger; the cost laws of Chapters 3 and 4 constrain specified events under hypotheses each theorem names. A conventional encoding can preserve those distinctions. A window that discards them cannot recover them in general. The stronger proposal that these distinctions belong in a necessary foundation of computation is a conjecture. The status of each result is stated below.

Abstract

This is a mathematical reference for the Thiele Machine: a deterministic transition system with instruction prices and a record that is a position in a preorder, with a toll on every step that moves the record out of the down-set of where it stood, a record no step lowers, and a designated start from which some run raises it. The one-bit record, a Boolean reading with its false-to-true crossing, is the two-point instance, and most results are stated on it. An internal ledger is an additional condition used by particular results. It states the model, its toll, the member predicate, the smallest member and the history lift, then the results, each over the class its premises name. Each result is stated with its premises and points to where it is argued.

A proof shows that conclusions follow from premises. It does *not* show that premises model physical reality. Every result is tagged as one of three types: **(S)** structural theorems about the model or a stated machine, **(C)** conditional results that hold under a named premise no machine fact discharges, and **(R)** refutations, where a theorem states the negation of a proposed claim and a named construction supplies the counterexample. One proposal, the pointer criterion of Chapter 9, has no proof; it carries a fourth tag, thesis. Results that are someone else's and argued in their own papers are tagged cited.

The structural results (S), by where they sit in the model:

- *The definition and the price in the move.* The member predicate: the toll, permanence and a run that makes a distinction; a flat reading obeys the toll and is not a member; the latch is the smallest member; every system with a Boolean equality on states and a run that ends off its start is the shadow of a member. The floor is the toll over any order of records, and the universal floor on the one-bit instance. The least instruction price and the least price for the lift. On finite machines, the toll derived from merge pricing for permanent certificates. Shadow pricing: no price computed from a window that hides a crossing is exact.

- *What stays.* A free move cannot touch a permanent reading. The record axis over any base is a latch, and a growing record decomposes into threshold latches.

- *What a window forgets.* The coarsest observational identification, and the factorization of histories through it. Shadow pricing, and what a window can decide and price.

- *The room a kept distinction needs.* On any state space a permanent flip merges or needs a fresh state every time, with the exact retention bound for a bounded memory, and the stochastic and unitary forms of the same count.

- *What no inside procedure decides.* The structural shortcut wall over any substrate with a recursion theorem; the recursion theorem and Rice's theorem (the logician Henry Rice, 1953: no program decides a property of what programs do, unless it holds of every program or of none) for L.

That a flat reading, a toggle and a one-state system are not members, and the counterexamples that show which premises do work, are refutations (R). Results stated over Landauer heat are conditional (C) on Landauer's principle, a named premise.

Contents

1 Introduction **2**

2 Notation and Conventions **3**

3 Thiele Machines and the Universal Floor **4**
3.1 Free moves, least prices and observation 6
3.2 A2 among cost frameworks 8
3.3 Several floors and one charge 8

4 Finite Machines and the Price of Merging **10**
4.1 Entropy and heat 13
4.2 An eight-state instance 13
4.3 No intrinsic joule value 14

5 Windows, Pricing and Narrowing **15**

6 Recursion and Undecidability **17**

7 The Record Axis **18**
7.1 Growing and probabilistic records 18
7.2 Bases 19
7.3 The record over any order 19

8 The Physical Boundary **20**

9 External Systems and the Pointer Thesis **21**
9.1 The pointer criterion 21

A Epistemological Classification **22**

Chapter 1

Introduction

A Thiele machine has states, moves, a price on each move and a record in a preorder; a move that raises the record pays (the toll), no move lowers it, and some run from a designated start raises it. A ledger, where a result uses one, is an additional condition, and a window that hides the record and the ledger is the subject of Chapter 5. The chapters below state that rule, derive its consequences, show which machines carry it, and prove where a window that hides the record and the ledger loses information.

Chapter 3 states the model, its toll, the member predicate, the smallest member, the history lift and the universal floor. Chapter 4 derives the toll on finite machines from the price of merging states and checks which premises are needed. Chapter 5 states what a window can decide and price. Chapter 6 states the recursion and undecidability results, over any substrate and for L. Chapter 7 states the record axis over any base and the latch decomposition of growing records. Chapter 8 states the physical boundary. Chapter 9 states the models of external systems and the pointer thesis.

Chapter 2

Notation and Conventions

- $\mathbb{N}$ is the natural numbers, $\mathbb{B} = \{\text{false}, \text{true}\}$, $\mathbb{R}$ the reals.
- A *trace* is a finite list of instructions. $\text{run}(\tau, s)$ folds the step function over τ from s.
- $(R, \leq)$ is the preorder of records and $\text{rec} : S \to R$ the record.
- P is the record on the two-point order, a reading from states to $\mathbb{B}$.
- $c(i) \in \mathbb{N}$ is the cost of instruction i; $c(\tau)$ is the sum over a trace.
- μ is the ledger. Where a state carries it, conservation means $\mu(\text{step}(s, i)) = \mu(s) + c(i)$.
- A *window* is a function from states to observations.

Chapter 3

Thiele Machines and the Universal Floor

Definition 3.1 (Priced record system)**.** A priced record system is a tuple $(S, I, \mathrm{step}, c, \mathrm{rec})$ with $\mathrm{step} : S \times I \to S$, $c : I \to \mathbb{N}$ and $\mathrm{rec} : S \to R$, where $(R, \leq)$ is a preorder given by a Boolean test. The record is a position in that order. A step from s by i *leaves the down-set* when $\mathrm{rec}(\mathrm{step}(s, i)) \not\leq \mathrm{rec}(s)$. The one rule is the toll, first named No Free Insight:

$$\textbf{A2:} \quad \forall s\, i,\ \mathrm{rec}(\mathrm{step}(s, i)) \not\leq \mathrm{rec}(s) \implies c(i) \geq 1.$$

The order carries no further assumption: not antisymmetry, not growth of the record along steps, not finiteness. An internal ledger is not part of this definition or the next; the results that use one say so.

Definition 3.2 (Thiele machine)**.** A Thiele machine is a priced record system together with a set of designated starts $\mathrm{starts} \subseteq S$ such that

- (toll) A2 holds;
- (permanence) $\forall s\, i,\ \mathrm{rec}(s) \leq \mathrm{rec}(\mathrm{step}(s, i))$;
- (distinction) for some $s_0 \in \mathrm{starts}$ and some trace τ, $\mathrm{rec}(\mathrm{run}(\tau, s_0)) \not\leq \mathrm{rec}(s_0)$.

A reading is *flat* when $\mathrm{rec}(\mathrm{step}(s, i)) \leq \mathrm{rec}(s)$ for every s and i. Flat is a property of the reading. In the other direction, a system with a Boolean equality on its states and a run that ends off its start is what some Thiele machine looks like through a window that drops the record (Theorem 3.2). Every Thiele machine has a move that costs at least 1 and has the floor over its order of Theorem 3.3 (Monograph, Theorem 2.3(2) and Observation 2.1).

Theorem 3.1 (The smallest Thiele machine)**.** ***(S)*** *The latch has two states* no $<$ yes*, one move that sends both to* yes *at price* 1*, and the state as its reading. It is a Thiele machine with start* no*, and its move is not injective: both states land on one. Every Thiele machine has two states with different readings; over the two-point order every Thiele machine has a move that raises the reading from some no-state and keeps every yes-state at yes; on two states read as themselves every Thiele machine has the latch's move; and the toggle, whose move swaps the two states, is not a Thiele machine.*

Argued in Monograph, Theorem 2.3(2)–(3). Its proof of (3) walks a run to the first move that lands on yes, which works for any member over the two-point order, not only on two states.

Theorem 3.2 (The history lift)**.** ***(S)*** *Let M be any tuple $(S, I, \mathrm{step}, c, \mathrm{rec})$ of that shape, toll or not, with a Boolean equality test on its states that agrees with equality, and x a state. The history lift of M at x has states $S \times \mathbb{B}$, the moves of M, step $((s, b), i) \mapsto (\mathrm{step}(s, i),\ b \vee [\mathrm{step}(s, i) = x])$,*

price 1 *on every move, and reading* $(s, b) \mapsto b$*. Dropping the bit returns the run of* M*. If some run from a state* $s_0 \neq x$ *ends at* x*, the lift is a Thiele machine whose starts are the states with bit false; so every* M *with a run that ends on a state other than its start is the shadow of a Thiele machine. Above every shadow sit two lifted states with different readings, and no function of the shadow equals the reading. The collision is between reachable states:* (x, false) *is a start, the run from* (s_0, false) *lands on* (x, true)*, and no function of the shadow equals the reading on the states runs reach from the starts. Over a finite base the least price list that meets the toll for the lift is* 1 *on every move that can land on* x *and* 0 *elsewhere, the toll holds for a price list exactly when it is at least that list, and the lift priced by that list is a Thiele machine.*

Argued in Monograph, Theorem 2.3(4), except the least price list.

Proof of the least price list. A move i raises the bit at some lifted state exactly when $\text{step}(s, i) = x$ for some s: from (s, false) the bit becomes $[\text{step}(s, i) = x]$, and from a raised bit nothing rises. Over the finite list $S \times \mathbb{B}$, Theorem 3.7 makes the schedule that is 1 on those moves and 0 elsewhere the least that meets A2, and every schedule that meets A2 lies above it. Permanence and the distinction do not mention prices, so the lift priced by that schedule is still a Thiele machine. □

The flat reading, and computation as a shadow. A reading is flat when no step leaves its down-set, so A2 holds with nothing to charge, no run of it leaves the down-set of its start, and it is not a Thiele machine for any choice of starts. A clock and a reversible process, each read through a reading its moves never raise, are flat, and so is any deterministic transition system (S, I, step) given a constant record: forget c and rec from any Thiele machine and what remains is flat. Flat is a property of the reading, not of the machine: every system with a Boolean equality test on its states and a run that ends on a state other than its start is the shadow of a Thiele machine (Theorem 3.2). Ordinary computation, read through a reading its moves never raise, is one flat shadow among others; read through whether it has halted, a halting computation with its step priced meets the definition on its own states (the halting theorem of Chapter 4). The theorems of this reference are about what is kept and what keeping it costs. Argued in Monograph, Theorems 2.3(1) and 1.2.

The one-bit instance. Take $R = \mathbb{B}$ with false $<$ true. The record is a reading $P : S \to \mathbb{B}$, leaving the down-set is a false-to-true crossing, and A2 reads

$$\forall s\, i,\ P(s) = \text{false} \wedge P(\text{step}(s, i)) = \text{true} \implies c(i) \geq 1.$$

Call a priced record system on this instance a certification system. The name does not restrict the interpretation of P: it is an arbitrary Boolean reading, and neither permanence nor the occurrence of both values is required. Most results below are stated on this instance, and each says so by writing P. A certification system obeys the toll on this instance; it is a Thiele machine only with permanence and a distinction added.

Theorem 3.3 (The floor is the toll)**.** ***(S)*** *For every priced record system, every run whose final record is not at or below its starting record costs at least* 1 *exactly when A2 holds, and under A2 every run costs at least the number of its steps that leave the down-set. When the record never drops along a step, leaving the down-set is a strict rise.*

Proof. A one-step run is a step, so the floor gives the toll. For the converse, induct on the run. If the first step stays in the down-set, the rest of the run ends outside the down-set of its own start, by transitivity of the order, and the induction hypothesis pays. If the first step leaves the down-set, A2 pays. Counting exits is the same induction. □

Also argued in Monograph, Observation 2.1 and Theorem 5.2.

Theorem 3.4 (Thresholds are one-bit machines)**.** ***(S)*** *For every point a of R, reading the record only as the bit "$a \leq \mathrm{rec}(s)$" gives a one-bit priced record system, and it obeys its toll whenever the original does; the original obeys A2 exactly when every such threshold does. When the order is antisymmetric, the thresholds determine the record. The one-bit floor of Theorem 3.9 is the two-point instance.*

Argued in Monograph, Theorems 5.2(3), 5.1(1) and 12.10(2).

Sliced interface. On the one-bit instance, a sliced system additionally has a surjection $\pi : S \to B$ and a partition $I = I_H \sqcup I_V$. Write $F_i(s) = \mathrm{step}(s, i)$. For every $i \in I_H$ there is $h_i : B \to B$ with

$$\pi(F_i(s)) = h_i(\pi(s)), \qquad P(F_i(s)) = P(s), \qquad c(i) = 0.$$

For every $v \in I_V$, require $\pi(F_v(s)) = \pi(s)$ and $P(s) \leq P(F_v(s))$, ordering false below true. A2 still applies. These equations hold on all states of the stated domain; they imply permanence. They do not require a product state space or a universal base.

For a non-certification instance, use $S = \mathbb{Z}_4 \times \{0, 1, 2\}$, $\pi(b, k) = b$ and $P(b, k) = [k = 2]$. The horizontal instruction $R(b, k) = (b + 1 \bmod 4, k)$ costs zero; the vertical instruction $V(b, k) = (b, \min(k+1, 2))$ costs one. Projection and reading preservation for R are immediate; V preserves the projection, never lowers the reading, and pays one on its crossing from $k = 1$ to $k = 2$. It also costs one at $k = 2$. The definition is Monograph, Definition 2.3; this example is its dial and slider.

Valuations. A Boolean valuation need not itself be an autonomous two-state quotient. Take three states, low, middle and high, one move sending low to middle, middle to high and high to high at price 1, and the reading "is it high". The toll and permanence hold. Low and middle both read false, but their next states read false and true, so no rule on the reading alone says what the next reading is.

3.1 Free moves, least prices and observation

Theorem 3.5 (All readings and free moves)**.** ***(S)*** *For a state type with decidable equality, fixed transitions and natural instruction prices, A2 holds for every Boolean reading if and only if every zero-cost instruction is the identity on all states.*

Proof. If a free move changes s to t, the decidable reading $[x = t]$ is false at s and true at t, contradicting A2. Conversely an identity move cannot raise any reading, and every positive natural price is at least one. □

Also argued in Monograph, Theorem 2.4.

This distinguishes a chosen priced reading from arbitrary state predicates; it does not select an autonomous base projection or a partition of moves.

Theorem 3.6 (Free reading preservation)**.** ***(S)*** *Under A2 and global permanence, every zero-cost instruction preserves the reading. With the other sliced-interface laws fixed, global permanence can replace horizontal reading preservation and vertical monotonicity.*

Proof. A free move cannot raise the reading by A2 and cannot lower it by permanence. Conversely, horizontal preservation and vertical monotonicity ensure that no instruction lowers the reading, since the two classes exhaust the instruction set. □

Also argued in Monograph, Theorem 2.5.

Theorem 3.7 (Least instruction toll)**.** ***(S)*** *Fix the transition and reading, and supply a finite list covering all states. For each instruction i, set $p(i) = 1$ if some state has a false-to-true crossing under i, and $p(i) = 0$ otherwise. Then A2 holds for a natural schedule c exactly when $p(i) \le c(i)$ for every i. The same equivalence holds for the global finite-trace crossing floor. Consequently p is the unique pointwise least valid schedule.*

Proof. Search the covering list. A crossing witness forces every valid instruction price to be at least one. Without a witness, zero meets every demand for that instruction. The resulting schedule is valid and no larger than any valid schedule. Antisymmetry proves uniqueness. A2 implies the trace floor by the first crossing, while one-step traces give its converse. □

Also argued in Monograph, Theorem 2.7. The finite list supplies an effective search; it is not a minimization algorithm for arbitrary infinite systems.

Theorem 3.8 (The coarsest observational identification)**.** ***(S)*** *For a total deterministic system with specified observation o, let $s \sim t$ mean $o(\mathrm{run}(w, s)) = o(\mathrm{run}(w, t))$ for every finite word w, including the empty word. This is the unique coarsest observation-preserving, step-stable equivalence. Any deterministic implementation preserving steps and observations must separate states that some future observation separates. The coarsest relations are pointwise equivalent.*

Proof. Equality for each word gives the equivalence laws. The empty word gives current-output preservation; prefixing a move gives step stability. Any stable output-preserving relation preserves outputs along a word by induction, so is contained in $\sim$. An implementation merging a separated pair would have to give two answers after the same continuation from one state. □

Uniqueness here is pointwise logical equivalence of relations. This is Edward Moore's distinguishing-experiment idea (1956) in its plain form.

Histories and maps. Give a pointed source machine an access word reaching each state. A start- and step-preserving map into a pointed target exists exactly when equal source endpoints of any two words imply equal target endpoints. Necessity follows by mapping the endpoints. For sufficiency, map a source state to the target endpoint of its access word; the implication makes this independent of all other words reaching it. The empty word and appending a move give the required laws, and reachability forces pointwise uniqueness.

If two reachable machines have the same initial output behavior and each is separated by future observations, this implication follows by appending all continuations and using target separation. With supplied access words for the first machine, the resulting map preserves outputs and is onto by target reachability and injective by source separation. It is pointwise unique. This is behavioral minimality, not a claim about smallest infinite cardinality.

Cost interfaces. The definition uses instruction costs $c : I \to \mathbb{N}$, so a trace's cost is independent of its initial state. Later source-charge results use a broader interface $a : S \to \mathbb{N}$ for one autonomous step, or $\widetilde{c} : S \times I \to \mathbb{N}$ with instructions. Their trace cost is the sum of charges along the actual run. The same first-crossing proof gives a floor if each crossing costs at least one, but instruction-priced theorems retain their stated signature and hypotheses. In particular, a single instruction with one fixed price cannot charge one at entry and zero at idle; a source charge can, by charging the state it moves from. Monograph, Section 8, works through both.

Theorem 3.9 (The universal floor)**.** ***(S)*** *In every one-bit system that obeys the toll, every one-bit Thiele machine among them, if $P(s_0) =$ false and $P(\mathrm{run}(\tau, s_0)) =$ true, then $c(\tau) \ge 1$. It is the two-point instance of the floor over any order.*

Proof. Induction on τ. Since P is false at the start and true at the end, some step in τ takes it from false to true. By A2 that step costs at least 1; every other step costs at least 0. □

Also argued in Monograph, Observation 2.2.

Theorem 3.10 (Simulating systems)**.** ***(S)*** *Let a certification system, the base, be simulated by another, the host: an embedding of base states into host states, a decoding of each base instruction into a list of host instructions, a commuting step law, and certification reflected by the embedding. A base trace from uncertified to certified costs at least* 1 *in the base, the host run of its decoded trace ends certified, and that host run costs at least* 1*.*

Proof. The base floor is Theorem 3.9. By the commuting law, block by block, the host run of the decoded trace from the embedded start ends at the embedding of the base's end state. Reflection makes the first uncertified and the second certified, so Theorem 3.9 on the host, under its own toll, charges that run at least 1. □

The commuting law compares endpoints of complete translated instruction blocks. The host's cost floor uses its own toll axiom. This theorem alone neither constrains intermediate block states nor establishes an earned history for arbitrary host programs. Such guarantees require preservation over the host's permitted operations and initial states. A host whose blocks pass through an uncertified, unpaid middle state still meets the commuting law, so the two claims are separate.

Theorem 3.11 (Quantitative form)**.** ***(S)*** *Extend a certification system with a witness* $w : S \to \mathbb{N}$ *and a threshold* k *such that each step raises* w *by at most its cost and every certified state has* $w \geq k$*. A trace from a state with* $w = 0$ *to a certified state costs at least* k*.*

Argued in Monograph, Theorem B.1.

Theorem 3.12 (A2 is a restriction)**.** ***(S)*** *A cost-bearing system without A2 can switch certification on at zero cost, and A2 excludes exactly that.*

Argued in Monograph, Theorem 5.5(3), the third way out; the second half is A2 restated.

3.2 A2 among cost frameworks

Theorem 3.13 (The toll as amortized cost)**.** ***(S)*** *A step system satisfies A2 if and only if every step has non-negative amortized cost under the potential that is* 1 *on uncertified states and* 0 *on certified ones; the universal floor is then the potential method's telescoping bound. Under A2 the number of certification flips along a trace is at most its cost, and A2 together with the ceiling that no trace costs more than its flips is equivalent to exact pricing of a permanent certificate: one unit per flip and nothing else.*

Argued in Monograph, Theorem 8.4, Observation 2.8, and the exact schedule in Section 8.

These are local correspondences. The cost-framework families they touch (graded monads, amortized resource analysis, cost translations, linear logic) are not embedded here.

3.3 Several floors and one charge

Theorem 3.14 (Joint floors)**.** ***(S)*** *For a finite family of events, the least cost schedule meeting every unit floor charges one on a step where any event occurs and zero otherwise. On pairs of Booleans, two independent coordinates that both rise in one step are met by a single charge of* 1*.*

Proof. Each floor forces at least one on every step where its event occurs, and nothing elsewhere. The schedule that is 1 where some event occurs and 0 elsewhere meets every floor, and it sits pointwise below every schedule that does. The Boolean pair is the case of two events. □

Independent state coordinates do not by themselves force additive cost.

Theorem 3.15 (Exact pricing)**.** ***(S)*** *For a price on each state and move, a schedule meets the certification floor and never overcharges (no run costs more than its flips) exactly when it charges on precisely the certification flips, one unit each, and nothing elsewhere.*

Proof. Apply both conditions to one-step runs. The floor charges every flip at least one; the ceiling charges every step at most its own flip count, one on a flip and zero elsewhere. Sums over longer runs follow. Monograph, Theorem 16.4, carries the same argument on gas. □

Chapter 4

Finite Machines and the Price of Merging

Fix a state type S with a finite enumeration, instructions I, a step function and a reading P. The certificate is *permanent* when no step switches it off. An instruction is *injective* when its step function is.

Theorem 4.1 (A permanent flip merges)**.** ***(S)*** *On a finite state space with a permanent certificate, an instruction that switches the certificate on at some state is not injective.*

Proof. If the instruction were injective it would map the certified states injectively into the certified states, so onto them, leaving no certified image for the uncertified state it switches on. □

Also argued in Monograph, Theorem 3.1.

This asserts non-injectivity of the instruction, not a collision at the entry destination. For $u \notin H$, $f(u) \in H$, and $f(H) \subseteq H$ with $|H| = K$, the path $u \to h_1 \to \cdots \to h_K \to h_K$ delays the orbit's collision until the $(K+1)$st transition. The entry shares its destination with a source in H exactly when $f(u) \in f(H)$. Surjectivity of $f|_H$ is sufficient, not necessary, for this condition at a particular entry. Instruction pricing charges every use of a non-injective instruction under the merge-pricing premise, regardless of where its collision occurs.

Theorem 4.2 (A2 from merge pricing)**.** ***(S)*** *On a finite state space with a permanent certificate, if every non-injective instruction costs at least* 1*, A2 holds, and so does the trace floor.*

Argued in Monograph, Observation 3.3: a flipping instruction merges, and merges are priced.

Each premise is needed. Unbounded history walks past finiteness by paying in fresh states, a new one every step; a revocable certificate keeps nothing; and an unpriced merge is a price list that skips the merge. On any state space, a raise that merges nothing sends the certified states one-to-one into a proper part of themselves, so it needs infinitely many of them. Argued in Monograph, Theorem 5.5 and Proposition 3.4.

Theorem 4.3 (The squeeze bound)**.** ***(S)*** *On a finite state space with a permanent certificate, with m certified states and k uncertified states that one instruction switches on, the instruction collapses at least k states. Priced one unit per halving of the number of distinct states, its cost c satisfies $m + k \le 2^c m$, so, when $m \ge 1$, $c \ge \log_2((m+k)/m)$; in rounded-up logarithms, $\lceil \log_2(m+k) \rceil \le c + \lceil \log_2 m \rceil$.*

Argued in Monograph, Theorem 3.10; the rounded form is Theorem 6.1 on a one-move run.

These bounds are tight, and A2 is equivalent to the floor. For every system, A2 holds exactly when every run from an uncertified state to a certified one costs at least 1. For every $m \ge 1$ and c, take m certified states, give each $2^c - 1$ uncertified states of its own, and let one move of cost c

send each block to its certified state: it flips $2^c m - m$ states, no landing spot takes more than 2^c, and the bound $m + k \leq 2^c m$ of the theorem above is met with equality. Its rounded-logarithm form is strictly weaker: $m = 3$, $k = 1$ and cost 0 meet the rounded form and fail $m + k \leq 2^c m$.

Theorem 4.4 (Which flips are forced)**.** ***(S)*** *When instructions have decidable equality, over all schedules that price merges, an instruction is forced to have positive price if and only if it merges two states. An instruction that switches a reading on either merges or switches it off somewhere else. So every permanent-record write is forced, and the only flips a merge-priced schedule can leave unpriced are revocable ones, which keep nothing.*

Argued in Monograph, Theorems 3.8 and 3.7 and Corollary 3.9.

Theorem 4.5 (Refutation)**.** ***(R)*** *"The forced events are exactly the permanent-record writes" is false: a three-state instruction flips, revokes, and merges at once.*

Argued in Monograph, Section 3, with the three cards T_0, T_1, T_2.

Theorem 4.6 (Merge, or a fresh state every step: the dichotomy)**.** ***(S)*** *On any state space, let a reading be permanent under a step f, and take a run $x_0, f(x_0), f(f(x_0)), \ldots$ in which the reading goes from false at step r to true at step $r + 1$. If no two states among the run's first n steps merge under f, those states are pairwise distinct; so on a finite state space of M elements, a run that keeps a record and never merges has fewer than M steps. In particular, for an injective move F that switches the reading on at s, the iterates of s are pairwise distinct and no finite duplicate-free list contains them all: a permanent flip either merges or visits a fresh state every time.*

Argued in Monograph, Proposition 3.5(1), Theorem 1.3 and Proposition 3.4.

Theorem 4.7 (Retention bound, and its exactness)**.** ***(S)*** *Let a finite implementation $F : X \to X$ of a reset carry an abstraction q that sends every image of F to a fixed abstract state q_0, with a representative of q_0 and representatives of $N - 1$ further, pairwise distinct abstract states, and let F be injective on the states reached from the representatives in fewer than T applications. Then $|X| \geq 1 + (T + 1)(N - 1)$; for $N \geq 2$, the history is therefore kept through at most $\lfloor (|X| - N)/(N - 1) \rfloor$ resets. For $N = 1$ the bound is only $|X| \geq 1$ and gives no finite retention horizon. The bound is attained: one fixed state and $N - 1$ paths of $T + 1$ states, with F walking each path and sending its last state to the fixed one, has exactly $1 + (T + 1)(N - 1)$ states, is closed under F, meets the reset and representative premises, is injective through T repetitions, and, when $N \geq 2$, merges on the next.*

Argued in Monograph, Proposition 3.5(2) and the exactness argument after it; the horizon is that bound solved for T.

Theorem 4.8 (The step law)**.** ***(S)*** *For any step f on any type with decidable equality and any start, every step of the run before its first revisit is fresh (the new state equals no earlier state of the run); the first revisit lands on the start or is a merge (it equals an earlier state x_j with $j \geq 1$ and the state before x_j differs from the current state, so two distinct states share an f-image); and every later step repeats a step the run already took, so the run is periodic and merges nothing new.*

Argued in Monograph, Theorem 1.1.

Theorem 4.9 (Refutation: the halting hole)**.** ***(R)*** *"Every raise of a permanent reading on finitely many states is a merge" is false for partial steps: on two states, $a \mapsto b$ with b having no successor raises the reading with no shared image. Totalising the halt ($b \mapsto b$) makes the raise a merge.*

Argued in Monograph, Theorem 1.2.

Theorem 4.10 (The write of a cell, and the fresh states it takes to avoid the merge)**.** ***(S)*** *On the states blank, zero, one with reading "definite," the write of a 0 is permanent, raises the reading from blank, and merges blank with zero. A fourth state remembering that the cell was blank does not avoid the merge: the write on four states still merges two distinct states. Only endlessly many fresh states avoid it, a new one every step: on the natural numbers the write "add one" is injective and never returns to 0; on the integers "add one" is a bijection, raises "at least one" from 0, keeps it, and after $k > 0$ steps is away from its start.*

Argued in Monograph, Theorem 1.4.

Theorem 4.11 (No state space codes all its own readings; the regress has a price)**.** ***(S)*** *For any type X and any coding $X \to (X \to \mathrm{bool})$, some reading $X \to \mathrm{bool}$ is coded by no state (Cantor). Under the halving price, a run that narrows N candidates to one costs at least $\lceil \log_2 N \rceil$ units, and for every m there is a candidate count whose rounded-up logarithm is at least m.*

Argued in Monograph, Theorems 1.5 and 6.1; unboundedness takes $N = 2^m$.

Theorem 4.12 (Nondeterministic merge)**.** ***(S)*** *Let a step relation on a finite duplicate-free list of states have a duplicate-free set Y of yes-states that is closed and total (every yes-state has a chosen successor in Y), and let some state outside Y relate to a state of Y. Then some state of Y has two distinct predecessors.*

Argued in Monograph, Section 21, where both nondeterministic arguments are printed.

Theorem 4.13 (Reversible keeps nothing)**.** ***(S)*** *For an injective step on a finite duplicate-free list containing every state, the run from any start returns to the start after a positive number of steps, and any reading that is false at the start and never goes from true to false is false at every step.*

Argued in Monograph, Theorem 1.6.

Theorem 4.14 (Noisy steps, both directions)**.** ***(S)*** *Let P be a nonnegative kernel on n states with rows summing to 1 and let Y be closed under P. If every column of P sums to 1, then $P(i,j) = 0$ for every $i \notin Y$, $j \in Y$. If some $i \notin Y$, $j \in Y$ has $P(i,j) > 0$, then some column sum differs from 1, and the scaled collision weight $\sum_j c_j^2$, with c_j the column sums, strictly exceeds n. For the uniform distribution $u_i = 1/n$, $c_j = n(uP)_j$, so the actual collision probability $\sum_j (uP)_j^2$ exceeds $1/n$, its initial value. Conversely, if every column sums to 1, then for every spread q, $\sum_j (qP)_j^2 \le \sum_i q_i^2$. Over the reals, for a strictly positive spread q on N states, $\sum_i q_i \ln(Nq_i) = \ln N - H(q) \ge 0$, with strict inequality as soon as some $q_i \neq 1/N$.*

Argued in Monograph, Theorem 1.7, whose proof of (2) also proves the entropy gap.

Theorem 4.15 (Unitary steps)**.** ***(S)*** *Let U be a unitary on a finite-dimensional inner-product space and Y a subspace with $U(Y) \subseteq Y$. Then $U(Y^\perp) \subseteq Y^\perp$. In projector form: if P is the orthogonal projection on Y and $(1-P)\,U\,P = 0$, then $P\,U\,(1-P) = 0$.*

Proof. U is injective and Y finite-dimensional, so $U(Y) = Y$. For $x \in Y^\perp$ and $y \in Y$ write $y = Uz$ with $z \in Y$; then $\langle Ux, y\rangle = \langle Ux, Uz\rangle = \langle x, z\rangle = 0$. Finite dimension enters once, in $U(Y) = Y$. □

Also argued in Monograph, Theorem 1.8.

Theorem 4.16 (Heat against time)**.** ***(S)*** *For $0 < p < 1$ the biased walk's entropy production per tick, $\sigma(p) = (2p-1)\ln\frac{p}{1-p}$, is 0 at $p = \frac{1}{2}$ and strictly positive otherwise. For $0 < v < 1$ and $N \ge 0$, $N \ln\frac{1+v}{1-v} \cdot \frac{N}{v} \ge N^2$.*

Argued in Monograph, Theorem 1.9. The displayed constant is deliberately weaker than the best bound for this formula. Indeed, the derivative of $\ln((1+v)/(1-v)) - 2v$ is $2v^2/(1-v^2) > 0$ and the expression is zero at $v = 0$. Thus the product is at least $2N^2$, strictly so for $N > 0$.

Remark 4.1. The monograph also develops four further arguments: one unit per crossing (a run costs at least its number of no-to-yes steps, with no permanence assumed, and that bound characterizes the toll; Observation 2.8); the certainty bound (under entropy pricing, the uncertainty about the start equals the part the run destroyed, at most its cost in bits, plus the part the end state still holds; Theorem 6.5); every scale paying the same way (if a coarse description of a machine merges, the fine step merges two described states, leaves the described set, or the described set is infinite; Theorem 1.10); and the device floor (under the stated thermal entropy-balance assumptions, a fixed protocol merging K inputs produces at least $k_B \ln K$ of entropy, at least $k_B T \ln K$ of dissipated work, on its worst input, however slowly it runs and whatever anyone knows; argued from a cited identity, below).

4.1 Entropy and heat

Theorem 4.17 (Entropy after a permanent step)**.** ***(S)*** *After a permanent step, any distribution on the m certified states and the k flipping ones has Shannon entropy at most $\log_2 m$. From the uniform distribution the step removes at least $\log_2((m+k)/m)$ bits. An injective step removes none.*

Argued in Monograph, Theorems 3.12 and 3.11.

Theorem 4.18 (Heat)**.** ***(C)*** *Given the named premise Landauer's principle, stated for a step and a distribution (Rolf Landauer argued at IBM in 1961 that a step that cannot be run backwards requires a minimal heat generation; the premise reads this as mean heat at least $k_B T \ln 2$ per bit of entropy the step removes, for the chosen physical memory and its input distribution, as an average over that preparation rather than a bound on each trial), the instruction has mean heat, when $k_B T \geq 0$, at least $k_B T \ln((m+k)/m)$ from the uniform distribution, and, when $k_B T > 0$, positive heat from any distribution that leaves every certified state and the flipping one possible. For a state known in advance, the premise is equivalent to heat ≥ 0, so it supplies no positive lower bound; it does not determine actual device heat, which depends on the physical process and preparation, and the ledger toll still charges its unit on a known run, since merge pricing prices the instruction whatever input was prepared.*

Argued in Monograph, Corollaries 3.13 and 3.14 and Theorem 3.17, each under Premise 3.5.

The monograph adds a floor that comes from the device rather than the premise. By the identity tying Ladyman's reading of Landauer's principle to Sagawa's, which is theirs and cited, not proved, and the second law, a fixed device that merges K inputs produces at least $k_B \ln K$ of entropy, at least $k_B T \ln K$ of dissipated work, on its worst input, however slowly it runs and whatever anyone knows. So on a known start the premise sets no floor, and the merging move still has one on its worst input.

4.2 An eight-state instance

Example 4.1. Four program slots and a certification flag make eight states. An advance step is injective; the certify step and every jump merge states. The cost rule prices exactly the merges, so A2 holds there as a theorem. Each fact is a direct check of the eight states; Monograph, Section 3, carries it.

4.3 No intrinsic joule value

Theorem 4.19 (Refutation)**.** ***(R)*** *For every state type and natural-number ledger, the energy assignments at scales 1 and 2 give values 1 and 2 respectively at every state whose ledger is 1. This is conditional on such a state; an identically zero ledger need not yield different energies. The ledger fixes no energy unit: at scale c the energy is $c\mu$, so at $\mu = 1$ it is 1 or 2. Choosing the scale $k_B T \ln 2$ gives one ledger unit that energy by definition; the scale is a premise, and nothing here measures it.*

Chapter 5

Windows, Pricing and Narrowing

Theorem 5.1 (Window adequacy)**.** ***(S)*** *A query on states can be answered by a function of a window only if it is constant on the window's fibers, the sets of states the window shows the same way. Given a representative for every view, fiber constancy is equivalent to factoring through a decoder.*

Argued in Monograph, Theorem 9.1.

Adequacy for a supplied operational model. Allow a relation of permitted successors, a chosen state view and required outputs. Supply a representative for each view and a permitted successor for each instruction and underlying state. A deterministic decoder and next-view rule exist exactly when equal views give equal current outputs and every permitted successor under a common instruction has the same view from equal-view sources. That is Theorem (Window adequacy) applied twice, once to the outputs and once to the views of the permitted successors, with the supplied representatives and successors. It does not establish that the supplied relation describes a physical apparatus, or infer effective choices from existential ones. Observable branching from the same state and instruction excludes an exact deterministic encoding preserving those outcomes, since a deterministic encoding gives one outcome per state and instruction.

Selection and finite validation. For the unpointed two-state toggle, every Boolean reading invariant under its state-exchange symmetry is constant: invariance gives $r(a) = r(b)$. Distinguished starts or task meanings are additional structure. For every finite horizon H a unit-priced clock raised at step 1 and reset at step $H + 1$ agrees through H with a latch raised at step 1, while both satisfy the toll at unit price and only the latch is permanent. These results constrain conclusions from bare dynamics and bounded tests; they neither refute every physical permanence model nor choose a universal physical description.

Theorem 5.2 (Shadow pricing)**.** ***(S)*** *If a certifying step and a non-certifying step show the same window before and after, no price computed from the window meets the certification floor without charging the non-certifying step. A window that shows the reading prices certification exactly.*

Argued in Monograph, Observation 9.2 and Theorem 13.1.

Theorem 5.3 (Retained history)**.** ***(S)*** *Keeping the previous state in a history makes every deterministic step injective, and the state part of the new pair is still the original step's result, so the observed transition is unchanged. The history takes a fresh state every step; it walks past the finiteness premise, not past what keeping takes.*

Argued in Monograph, Theorem 18.2.

Theorem 5.4 (Narrowing)**.** ***(S)*** *On a finite machine priced one unit per halving of the number of distinct states, each halving of the machine's own set of possible states along a run is paid for. What an observer knows can shrink at zero cost: a measurement that copies a hidden bit into a blank display forgets nothing, borrowing the blank display (fresh states), and one admissible merge-priced cost assigns it zero; wiping the display is a merge and costs at least one. The WIPE charge is an instruction-level bound over all its possible sources. On the particular correlated pair produced by copying, the hidden bit can retain the entire distinction after WIPE. A charge of one is therefore not a claim that this preparation loses one bit of joint entropy, or dissipates one corresponding heat unit.*

Argued in Monograph, Theorems 6.1 and 6.2.

Chapter 6

Recursion and Undecidability

Definition 6.1 (Substrate)**.** A substrate supplies states, programs, a partial run function, a ledger that no step decreases, an encoding of programs into states with a decoder, behavioral equivalence (equal runs from every state), a representability predicate on program transformers, and the recursion theorem stated and proved by the logician Stephen Kleene in 1938, for representable transformers, as a premise. A shortcut predicate on programs respects behavioral equivalence and has one accepted and one rejected canonical program. This is Monograph, Definition 14.1.

Theorem 6.1 (Structural undecidability)**.** ***(S)*** *In every substrate with a shortcut predicate, no decider whose flip transformer is representable returns true exactly when the predicate holds.*

Proof. Let D be such a decider and $f(p)$ the rejected canonical program if $D(p)$ is true and the accepted one otherwise. The recursion theorem gives q behaviorally equivalent to $f(q)$. If $D(q)$ is true, q behaves like the rejected program, so the predicate fails at q; if false, it holds. Either way D is wrong at q. □

Also argued in Monograph, Theorem 14.1.

Theorem 6.2 (L)**.** ***(cited)*** *For the weak call-by-value lambda calculus L, Kleene's second recursion theorem holds, every extensional nontrivial property of L terms is undecidable, and L halting is undecidable.*

These are not argued here. L is the calculus Yannick Forster and Gert Smolka presented as a model of computation (Interactive Theorem Proving, ITP 2017, LNCS 10499), and the arguments are theirs, following the proofs for the full lambda calculus in Henk Barendregt's textbook (1984). That L and Turing machines simulate each other is Forster, Kunze, Smolka and Wuttke's (ITP 2021).

The diagonal uses recursion and behavioral equivalence; cost plays no part. Deciding a predicate over arbitrary programs is a different task from reading one field of a given state.

Chapter 7

The Record Axis

Definition 7.1 (Bases and base-driven records)**.** A base is a deterministic record-free machine: states, a next function, initial states and halting. A record-carrying machine has states, a next function, initial states, a record, a ledger and halting. A cover maps it onto a base step for step, preserving initial states and halting. The extension is *base-driven* when its record's next value depends only on the base state and itself, the ledger never decreases, A2 holds, the record is permanent, and some reachable step writes it. This is Monograph, Definition 12.1.

Theorem 7.1 (The record axis is a latch)**.** ***(S)*** *Every base-driven record over every base factors as a latch: there is a base event h such that the base state and the record evolve as $(b, r) \mapsto (\mathrm{next}(b), r \vee h(b))$. Two permanent records factor as two latches, each reading the other.*

Argued in Monograph, Observation 12.1 and Lemma 12.4.

Theorem 7.2 (Both premises do work)**.** ***(R)*** *A record that can switch back off is not a latch. A record switched on by a hidden clock is permanent and not driven by the computation.*

Argued in Monograph, Lemmas 12.5 and 12.6.

Theorem 7.3 (Which machines carry the axis)**.** ***(S)*** *Every base carries it: the base plus a latch on any event it reaches, charging one unit when the latch sets, is a base-driven record. A reversible base with unbounded memory carries it reversibly. With finite memory, a step that writes a permanent record is not injective.*

Argued in Monograph, Theorems 12.2 and 12.3.

7.1 Growing and probabilistic records

Theorem 7.4 (Growing records as families of latches)**.** ***(S)*** *A record whose next value is a function of the base state and its current value, that only grows in a partial order, whose ledger charges every strict change, and that changes at some reachable step satisfies a complete family of threshold-latch equations and keeps its price schedule. A monotone chain of pairwise-distinct k-bit vectors has at most $k + 1$ members.*

Argued in Monograph, Theorem 12.10(1), (3) and (5).

Theorem 7.5 (Limits of one latch and a fixed schedule)**.** ***(R)*** *One Boolean latch does not carry every growing record: a three-value chain over a one-state base is a counterexample. In a finite branching model a deterministic latch does not handle branching, and equal branch support and charges do not determine the branch weights.*

Argued in Monograph, Theorems 12.10(4) and 12.9.

7.2 Bases

Call two bases weakly equivalent when some relation relates every start of each to a start of the other, related states agree on the chosen observations and on halting, and a step of either is matched by finitely many steps of the other, possibly none, ending in related states. That is an equivalence: the identity relation witnesses reflexivity, the converse of a witness witnesses symmetry, and the composite of two witnesses witnesses transitivity, since finitely many matching steps of finitely many steps are finitely many. The record axis is a latch over the Cook and Reckhow RAM, the unit-cost random-access machine Stephen Cook and Robert Reckhow defined to measure time bounds (Journal of Computer and System Sciences 7, 1973), and over L; these instances follow from the general theorem (Monograph, Observation 12.1, over any base) and do not distinguish the bases. Cost-respecting simulations between different bases are not constructed.

7.3 The record over any order

The definition of Chapter 3 has the record in any preorder, and the theorems there state its floor and its thresholds. This section states the further results on that axis.

Theorem 7.6 (Latches, merges and windows on the axis)**.** ***(S)*** *A record that never drops and whose next value is a function of the base state and the current record factors through thresholds: some event h makes a at or below the next record exactly when a is at or below the current record or h fires. A repeat-free chain of k-bit words, each word's set bits among the next word's, has at most $k+1$ members, and some chain has exactly $k+1$. With finitely many states and a growing record, every instruction that leaves the down-set from some state merges two states, and pricing every merge gives the toll. For finitely many states and moves and decidable observations, some price computed from a window is exact exactly when the window has no collision.*

Argued in Monograph, Theorems 12.10(1) (its proof uses only that the record is driven and grows, so it holds over any preorder), 5.3(1), 5.4(2)–(3) and 13.1(2).

Theorem 7.7 (Entropy on the axis)**.** ***(S)*** *Under the halving price, on a state space of any size, for a non-negative weighting and any finite list of landing spots, each given with a repeat-free list of exactly the states the move sends to it, the move's conditional entropy loss over those lists is at most their total weight times the move's cost times* $\ln 2$*.*

Proof. Under the halving price a landing spot with n states in its list has $n \le 2^c$, with c the move's cost (Monograph, Theorem 13.2(4)). The entropy lost at a spot of weight w, spread over its n states, is at most $w \ln n \le w\, c \ln 2$. Sum over the spots. The finite, normalized form is Monograph, Theorem 3.16. □

Chapter 8

The Physical Boundary

Theorem 8.1 (Calorimeter)**.** ***(S)*** *Take two levels, ground at energy* 0 *and excited at* Δ*. The canonical reset is the one-step master equation whose matrix sends excited to ground with probability one and keeps ground at ground. From the half-excited populations* $(\frac{1}{2}, \frac{1}{2})$ *it reaches* $(1, 0)$ *exactly, and at fixed Hamiltonian the bath receives* $\Delta/2$*.*

Proof. The matrix applied to $(\frac{1}{2}, \frac{1}{2})$ gives $(1, 0)$. With the energies held fixed, the heat to the bath is the drop in the system's mean energy, from $\Delta/2$ to 0. □

Theorem 8.2 (Refutation)**.** ***(R)*** *Population dynamics do not fix heat: two gaps with identical dynamics release different heat.*

Proof. Run the same matrix with gaps $\Delta_1 \neq \Delta_2$. The populations move identically, and the heats are $\Delta_1/2 \neq \Delta_2/2$. □

Monograph, Section 18, sets this beside the ledger.

Ruling a gap out needs a thermal premise such as local detailed balance, or device evidence. Converting the ledger to energy requires thermal and device premises; the ledger has no intrinsic energy unit (Chapter 4).

Chapter 9

External Systems and the Pointer Thesis

Each model below is narrow. None is a full security, consensus, database or audit-log theorem.

- **RFC 9162 (Certificate Transparency).** The iterative inclusion and consistency checks of the specification reject the boundary cases and accept the specification's examples under symbolic digests. Collision resistance and signed-tree-head authenticity are cryptographic premises; neither is proved. Argued in Monograph, Definition 16.5 and Theorem 16.7.
- **Proof-carrying code.** A toy checker written as the address-bound verification condition accepts exactly when that condition holds; that is how it is defined, not a finding. It mirrors the roles in George C. Necula's proof-carrying code (POPL 1997), where a program ships with a proof that it is safe; the full system is outside this model.
- **Trusted Platform Module (TPM) quotes.** A bare sign/verify interface does not imply quote authenticity: a verification check that says yes to everything meets the interface and authenticates nothing. That a quote cannot attest state nobody measured is Monograph, Theorem 16.6.
- **Casper FFG.** Casper FFG is the finality gadget Vitalik Buterin and Virgil Griffith proposed for Ethereum in 2017. Two finalized blocks on conflicting branches imply a slashable quorum. This is the accountable-safety theorem of their paper, argued in Monograph, Theorem 16.1.

The addressed-RAM and reversible-update models are stated only for their named fragments (Monograph, Section 16). The cost-semantics correspondences are not full cost-framework embeddings.

9.1 The pointer criterion

The pointer-observable criterion proposes that the event a step rule is forced to price is singled out by redundant record proliferation across observers (Monograph, Section 17). Two things can be said for certain. Consensus, authenticity and updates without a coordinator do not force permanence: two observers who both see an event that flips every step agree, report truly, and watch it revoke. Durable views do imply permanence, because a view that stays true once true is permanence taken as a premise. The pointer criterion is a thesis: "faithfully model" and "deployed" tie it to the world and cannot be defined, as in the Church–Turing thesis.

Appendix A

Epistemological Classification

Result	Status	Key premise
The definition and the price in the move		
The member predicate; a flat reading obeys the toll and is not a member	**(S)**	Any preorder; flat means no step leaves the down-set
The latch is the smallest Thiele machine	**(S)**	Two states, one move priced 1; any member has two readings and a priced move
The history lift: a machine with a run that moves is the shadow of a member	**(S)**	A Boolean equality test on states; a run ending off its start; a finite base for the least price
The floor is the toll	**(S)**	A preorder of records; nothing else
Thresholds are one-bit machines	**(S)**	A2; antisymmetry for the thresholds to determine the record
Universal floor	**(S)**	A2 on the two-point order
All readings and free moves	**(S)**	Decidable state equality; one schedule for every Boolean reading
Least instruction price	**(S)**	Fixed dynamics and reading; finite covering list
Least unit lift price	**(S)**	An event witness for every mark; all-state toll
A2 from merge pricing	**(S)**	Finite states, permanent certificate, merge price
Shadow pricing	**(S)**	A colliding pair in the window
What stays		
Free reading preservation	**(S)**	A2 and global permanence
Record axis is a latch	**(S)**	Base-driven record
What a window forgets		
Canonical future equivalence	**(S)**	Fixed deterministic operations and observations
The room a kept distinction needs		
Merge or fresh states	**(S)**	Total deterministic step, permanent raise
Sharp retention bound	**(S)**	Finite implementation, distinct representatives, reset and injectivity through the specified horizon
Stochastic and unitary holding	**(S)**	Finite stochastic kernel with column constraints, or finite unitary dynamics and invariant projection
What no inside procedure decides		
Structural undecidability	**(S)**	Recursion theorem of the substrate
L recursion and Rice	cited	Forster and Smolka's L
The world		
Biased-walk heat against time	**(S)**	Specified biased-walk entropy formula; positive speed below one
Heat floor	**(C)**	Landauer's principle as a premise, finite states, permanent certificate, the uniform distribution on the states in play

No intrinsic joule value	**(R)**	None beyond the ledger
Calorimeter heat scale	**(R)**	Fixed dynamics
Pointer criterion	Thesis	Observer model